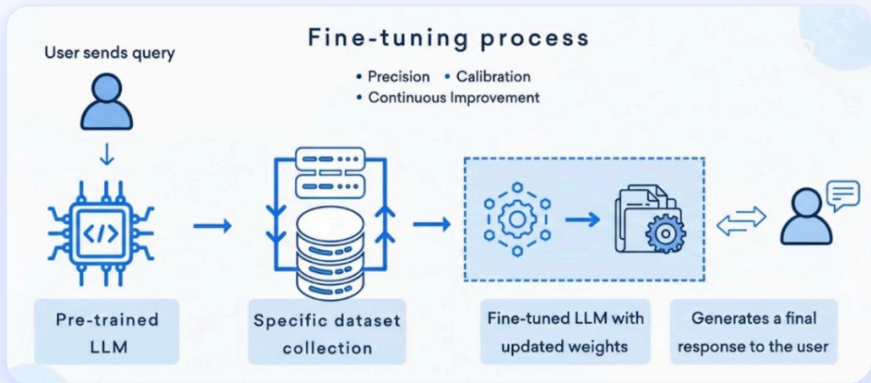


COMPLETE GUIDE

Fine Tuning Explained

Everything you need to know about adapting pretrained models



Naved Khan
Senior Gen-AI Engineer

+ Follow

What is Fine Tuning?

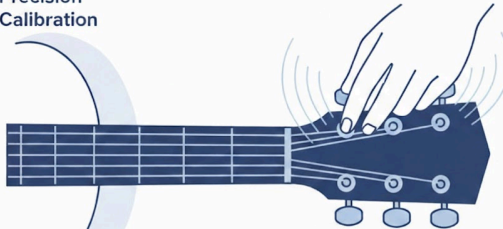
Fine tuning adapts a pretrained model to a specific task using smaller specialized datasets.

You take a model trained on general data. Then you train it more on your specific data. This updates the model's weights to better handle your task.

Think of it like tuning a guitar. The guitar already knows how to make music. You just adjust the strings to play specific songs. The guitarist fine-tunes each string for perfect pitch without rebuilding the instrument.

Fine Tuning!

- Precision
- Calibration



Naved Khan
Senior Gen-AI Engineer

+ Follow

Fine Tuning vs Pretraining vs Transfer Learning

Pretraining

Initial training on large unlabeled data. Model learns general features. Starts from random weights. Very expensive.

Transfer Learning

Broader concept. Taking knowledge from one problem to another. Includes pretraining and adaptation phases.

Fine Tuning

Type of transfer learning. Loads pretrained weights. Trains further on task specific labeled data. Updates weights for your task.

Fine tuning is a specific form of transfer learning that follows pretraining.



Naved Khan
Senior Gen-AI Engineer

+ Follow

Full Fine Tuning

Full fine tuning updates all model parameters during training.

Every weight in the model gets updated. You start with pretrained weights. Then you train on your data. All parameters change.

✓ Pros:

- ▶ Maximum adaptation to your task
- ▶ Best performance potential
- ▶ Complete model customization

✗ Cons:

- ▶ High memory requirements
- ▶ Expensive computational cost
- ▶ Risk of overfitting

Best for: When you have abundant resources and data



Naved Khan
Senior Gen-AI Engineer

+ Follow

LoRA - Parameter-Efficient Fine Tuning

LoRA adds trainable low-rank matrices while freezing most parameters.

Instead of updating all weights, LoRA adds small matrices. These matrices adapt to your task. The original weights stay frozen. This reduces trainable parameters dramatically.

Think of it like adding adapters to a machine. The machine stays the same. But adapters let it work differently. Much cheaper than rebuilding the machine.

Key Benefits:

- ▶ 99% parameter reduction. Example: 15M to 65K
- ▶ Memory efficient. Less GPU needed
- ▶ Faster training. Maintains performance



Naved Khan

Senior Gen-AI Engineer

+ Follow

LoRA Implementation

Here's how to implement LoRA:

PYTHON

```
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("model_name")

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    task_type=TaskType.CAUSAL_LM,
    bias="none"
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
```

Key Parameters:

- ▶ **r**: Rank. Lower = fewer params
- ▶ **lora_alpha**: Scaling factor. Usually 2x rank
- ▶ **target_modules**: Which layers to adapt
- ▶ **task_type**: Task type. CAUSAL_LM for language modeling
- ▶ **bias**: Bias handling. "none" = don't train biases



Naved Khan
Senior Gen-AI Engineer

[+ Follow](#)

QLoRA - Quantized LoRA

QLoRA combines LoRA with 4-bit quantization for even more memory savings.

Frozen model parameters stay in 4-bit precision. Only low-rank matrices update during training. This cuts memory usage dramatically.

Think of it like compressing a photo. The photo stays usable. But it takes way less space. QLoRA does this for model weights.

Memory Savings:

- ▶ Enables 7B models in under 10GB GPU memory
- ▶ 4-bit quantization reduces size by 4x
- ▶ Makes fine-tuning accessible on consumer hardware

Game changer for fine-tuning large models on limited resources.



Naved Khan
Senior Gen-AI Engineer

+ Follow

Fine Tuning Process

Fine tuning follows a structured workflow from data to deployment.

- ▶ **1. Dataset Preparation:** Curate quality data for your task
- ▶ **2. Model Selection:** Choose base model that fits your needs
- ▶ **3. Training Configuration:** Set hyperparameters and method
- ▶ **4. Training Execution:** Run training on your dataset
- ▶ **5. Evaluation:** Test performance on validation set
- ▶ **6. Iteration:** Adjust and retrain based on results

Each step matters. Quality data leads to quality results.



Naved Khan
Senior Gen-AI Engineer

+ Follow

Dataset Preparation

Here's how to prepare your dataset:

PYTHON

```
from datasets import Dataset
from transformers import AutoTokenizer

# Load and format data
data = [
    {"instruction": "...", "output": "..."}
]
dataset = Dataset.from_list(data)

# Tokenize for training
tokenizer = AutoTokenizer.from_pretrained("model_name")

tokenized = dataset.map(
    lambda x: tokenizer(x["instruction"]),
    batched=True
)
```

Best Practices:

- ▶ Use consistent formatting across examples
- ▶ Include diverse examples for better generalization



Naved Khan
Senior Gen-AI Engineer

[+ Follow](#)

Real-World Examples

Cosine - Software Engineering

Fine-tuned GPT-4o on real software engineer examples. Achieved 43.8% score on SWE-bench Verified Benchmark. Model autonomously identifies bugs, builds features, and refactors code.

Distyl - Text-to-SQL

Fine-tuned GPT-4o for text-to-SQL tasks. Secured first place on BIRD-SQL benchmark. Achieved 71.83% accuracy. Model converts natural language to SQL queries effectively.

OpenAI GPT-4o

OpenAI allows fine-tuning GPT-4o with custom datasets. Improves performance for specific use cases. Reduces costs. Can produce strong results with just a few dozen examples.

Use cases: coding, creative writing, function calling, NER tasks



Naved Khan
Senior Gen-AI Engineer

+ Follow

Fine Tuning vs RAG

Fine Tuning

Updates model weights. Domain-specific knowledge baked into model. Model learns your style and domain. Requires training data. Changes model permanently.

RAG (Retrieval Augmented Generation)

Connects to external knowledge sources. No weight updates. Dynamic knowledge retrieval. Uses vector databases. Model stays unchanged.

When to Use Each:

- ▶ **Fine Tuning:** Style, tone, domain-specific patterns
- ▶ **RAG:** Dynamic knowledge, frequently changing data
- ▶ **Hybrid:** Combine both for optimal results



Naved Khan
Senior Gen-AI Engineer

+ Follow

Fine Tuning Best Practices

- ▶ **Quality Data:** Curate high-quality examples. Quality beats quantity.
- ▶ **Proper Formatting:** Use consistent structure across all examples.
- ▶ **Hyperparameter Tuning:** Start with defaults. Adjust based on results.
- ▶ **Evaluation:** Test on validation set. Measure real performance.
- ▶ **Iterative Optimization:** Train, evaluate, adjust, repeat.
- ▶ **Avoid Overfitting:** Use validation set. Stop early if needed.

Remember: Good fine-tuning requires patience and iteration.



Naved Khan
Senior Gen-AI Engineer

+ Follow

**Want more content like
this?**



Naved Khan

Senior Gen-AI Engineer



Like



Comment



Share



Save

Follow for more AI and fine-tuning insights